Test corpus: what we validate against, and what we deliberately do not

The parsers in this repo are only as trustworthy as the input they have met. This records where that input comes from, and one thing we decided **not** to use.

What we use

1. Reference archives from SngCli

`internal/sng/testdata/reference-sngcli-v0.3.0.sng` was produced by **SngCli v0.3.0** (MIT, `mdsitton/SngFileFormat`) from a song folder we wrote. It is the only fixture that checks the reader against something other than our own understanding of the format — the fixtures in `mask_test.go` and `read_test.go` are built by an encoder written from the same reading of the spec as the reader, and two components sharing one misreading agree with each other perfectly.

SngCli is installed at `%LOCALAPPDATA%\Programs\sngcli\win-x64\SngCli.exe`. To regenerate:

```
SngCli.exe encode -i <folder-of-song-folders> -o <out> --noStatusBar -t 1
SngCli.exe decode -i <folder-of-sng-files>    -o <out> --noStatusBar -t 1
```

`.gitignore` excludes `*.sng` so nobody commits a song library, with an explicit exception for `internal/sng/testdata/`.

2. YARG itself, as the end-to-end oracle

YARG v0.15.0 is installed portable at `%LOCALAPPDATA%\Programs\YARG\YARG.exe` — the official Windows x64 zip, no installer, no elevation, deletable in one command. LGPL-3.0-or-later, free, and downloaded from YARC's own release page.

This is the gate that matters for the writer: put a repacked `.sng` in a songs folder, let YARG scan it, and confirm it appears with the right metadata and is playable. Nothing we can assert about our own output substitutes for the client accepting it.

(YARC publishes a `.sig` beside each release asset but no plain checksum, and no public key we could find, so the download is recorded by its SHA-256 rather than verified: `a2eac765a190e34a8acbda5b1351844704b427349d6d36b8cf1096f575055e41`.)

3. Charts we author

Hand-written `song.ini` and chart files exercising the messy cases: BOMs, Latin-1 bytes, duplicate keys, missing section headers, values containing `=`, free-text years, unknown keys, absurd numbers. These are in the unit tests.

What we deliberately do not use

YARG's Official Setlist

It would be the obvious corpus — real charts, real metadata, distributed by YARC themselves — and we are **not** using it.

`YARC-Official/Official-Setlist-Public` states plainly that the setlist is proprietary, that "musicians gave permission to use these songs in YARG specifically", that everything "**must** be used exclusively for YARG", and that users should not download it manually. Using it as the test corpus for a third-party song server is outside that permission — and serving it from one would be squarely outside it.

That is not a close call, and it matters more here than it would elsewhere: this project's own goal is to make charts distributable *separately* from music precisely to avoid copyright problems. A project with that goal cannot be casual about the provenance of its own test data.

The same caution applies to bulk-downloading community charts from Chorus Encore or similar: individual charters' licensing varies, and most community packages bundle commercial audio. Individual charts may be used where the charter's own licence permits it — that is a per-chart question, answered before the chart is added, not a bulk decision.

What the oracle actually found, 2026-09-05

`cmd/mkcorpus` wrote 20 deliberately awkward song folders; YARG v0.15.0 scanned them; its `badsongs.txt` and song cache were read back. **Our scanner had passed all 20 and was wrong about three of them.** These are the only findings in this repo that came from an oracle rather than from our own reasoning, which is precisely why they are the ones worth having.

Case	YARG	We had	Fixed
`song.ini` with no `[Song]` header	reads **nothing**, titles the song after its folder	read the keys	yes — parser now requires the header, and the song is flagged `no_metadata_section`

Case	YARG	We had	Fixed
chart with no audio	rejects: "*No audio accompanying the chart file*"	accepted it silently	yes — flagged `no_audio`
folder with a chart but no `song.ini`	neither cached nor reported bad — silently skipped	accepted it silently	yes — flagged `no_song_ini`

The headerless case is the one that mattered most. Our leniency was a guess written into a comment ("some charts omit it"), and it would have produced a catalog whose titles disagreed with what the player sees on their own screen — a bug with no error message anywhere.

Confirmations, where YARG agreed with decisions we had reached from source:

- Duplicate keys resolve **last-wins** — `FIRST` never appears in YARG's cache, `SECOND` does. That is now three independent sources: `IniModifierCollection.cs`, SngCli's round-trip, YARG.
- Latin-1, UTF-16LE and UTF-8-BOM `song.ini` files all read correctly on both sides.
- `notes.mid` is a **hard selection** over `notes.chart`, not a preference. Given both, YARG chose the `.mid`, found it unplayable, and rejected the whole song rather than falling back. Hashing the wrong file would be a real divergence, exactly as `chartfile.go` warns.
- Uppercase `[SONG]`, ragged whitespace, CRLF, values containing `=`, free-text years, unknown keys, the `cover` override, clean/explicit stem variants and a 4-way drum kit all matched.

Two divergences left open, deliberately

1. **UltraStar `notes.txt`.** YARG rejected it with "*Name metadata not provided*" even though the `song.ini` carried `name = UltraStar`. So UltraStar charts appear to take their title from somewhere other than `song.ini`. We report the `song.ini` name. **Not guessed at** — the rule is not understood, and inventing one would be worse than a known gap. Settle it by reading `ScanUltraStar` before UltraStar support is claimed.

2. **We do not know whether a chart is playable.** YARG rejected a header-only `.mid` with "*No notes found*"; we catalogued it happily. That is expected — deriving what a chart contains is Phase 1 step 6, the MIDI preparsers — but until then the catalog can offer a song the client will refuse, and that limit should be stated rather than discovered by a user.

The writer's gate, 2026-09-05

The same oracle was then pointed at output we produced. `yarg-song-server pack` repacked 14 corpus folders plus the reference song; YARG scanned the 15 archives and accepted 14, reading every title out of our metadata section.

The single rejection — `No notes found` — was settled by a **control** rather than explained away: SngCli's own archive of the same source folder was placed beside ours and scanned in the same pass, and YARG rejected both with the identical error. The fixture's hand-written chart has no note events. Without that control, "YARG rejected one of ours" would have looked like a writer bug, and chasing it would have been wasted work.

Independently, `SngCli decode` extracted our archive with zero errors and every contained file came back byte-identical to the source folder.

Reproducing this

```
go run ./cmd/mkcorpus -out $env:USERPROFILE\yarg-test\corpus
# point YARG at it by editing SongFolders in
#   %USERPROFILE%\AppData\LocalLow\YARC\YARG
elease\settings.json
# then delete songcache.bin, launch YARG, and read badsongs.txt beside it
```

The corpus is generated, never committed: it is large, and a committed corpus rots into something nobody regenerates.

The gap this leaves, stated honestly

Self-authored input cannot produce unknown unknowns. We can write a `song.ini` with the mistakes we thought of; we cannot write one with the mistakes we did not. Two things partly close that gap:

- SngCli's output already taught us two things synthetic input never would — that duplicate keys resolve last-wins in the reference tool, and that a contained file's extension can lie about its container.
- YARG scanning our output is a real oracle, because it is the actual client with the actual parser.

Neither is a substitute for a large real library. If a legitimately-licensed corpus becomes available, revisit this.

That said, the first run of this method found three real bugs, two of which had been written into the code as confident comments. The oracle is doing work.